

```
<span class="font-medium">
"€"
"89.92"
</span>
```

BINGO! Genau das ist es. Da haben wir den Beweis für das gefürchtete **Split-Node-Problem**.

Wenn in einer React-App der Code im Hintergrund so aussieht: `<span>{"€"}{price}</span>`, dann rendert React das im DOM als **zwei völlig getrennte Text-Bausteine** (Text-Nodes), die zufällig nebeneinander liegen.

Unser regulärer Ausdruck hat bisher in jedem Text-Node einzeln nach €89.92 gesucht. Da der eine Node aber nur € und der andere nur 89.92 enthält, schlägt die Suche immer fehl.

Hier ist der finale Fix. Wir bringen unserem Skript jetzt bei: *"Wenn du einen Text-Node findest, der nur aus einem € besteht, dann schau dir deinen direkten Nachbarn an. Wenn der Nachbar eine Zahl ist, manipulierte beide!"*

Hier ist das **komplette, korrigierte Skript**, das dieses React-Verhalten überlistet:

JavaScript

```
(function() {
  'use strict';

  // 1. Einfache Übersetzungen (Case-Insensitive durch Regex)
  const textTranslations = {
    "new subscription": "Neues Abonnement",
    "active": "Aktiv",
    "started on": "Gestartet am",
    "skipped until": "Übersprungen bis",
    "skipped": "Übersprungen",
    "paused until": "Pausiert bis",
    "paused": "Pausiert",
    "weeks": "Wochen"
  };

  // 2. Erweitertes Monats-Wörterbuch
  const monthMap = {
    "jan": "Jan.", "january": "Januar",
```

```

    "feb": "Feb.", "february": "Februar",
    "mar": "März", "march": "März",
    "apr": "Apr.", "april": "April",
    "may": "Mai",
    "jun": "Juni", "june": "Juni",
    "jul": "Juli", "july": "Juli",
    "aug": "Aug.", "august": "August",
    "sep": "Sep.", "september": "September",
    "oct": "Okt.", "october": "Oktober",
    "nov": "Nov.", "november": "November",
    "dec": "Dez.", "december": "Dezember"
  };

```

```

function processTextNode(node) {
  let text = node.nodeValue;
  let originalText = text;

```

```

  // --- A. WÄHRUNGSFORMAT (React Split-Node Fix) ---
  // Fall 1: React hat "€" und "89.92" in zwei Nodes aufgeteilt
  if (text.trim() === "€") {
    let nextNode = node.nextSibling;

```

```

    // Falls es leere Text-Nodes dazwischen gibt (z.B. durch Zeilenumbrüche), überspringen wir diese
    while (nextNode && nextNode.nodeType === 3 && nextNode.nodeValue.trim() === "") {
      nextNode = nextNode.nextSibling;
    }

```

```

    // Wenn der Nachbar-Node existiert und die Zahl enthält
    if (nextNode && nextNode.nodeType === 3) {
      const nextText = nextNode.nodeValue;
      const numberRegex = /^s*(\d+)\.(\d{2})s*$/;

```

```

      if (numberRegex.test(nextText)) {
        // 1. Das "€" im aktuellen Node löschen (behält aber Abstände)
        node.nodeValue = text.replace('€', '');

```

```

        // 2. Im nächsten Node aus "89.92" -> "89,92 €" machen
        nextNode.nodeValue = nextText.replace(numberRegex, '$1,$2 €');
        return; // Beenden, wir haben beide Nodes erfolgreich verarztet
      }
    }
  }
}

```

```

// Fall 2: Fallback, falls "€89.92" doch mal zusammen in einem Node stehen
text = text.replace(/€s*(\d+)\.(\d{2})/g, '$1,$2 €');

// --- B. DATUMSFORMAT ---
// "May 15, 2026" oder "Jun 12, 2026"
text = text.replace(/\b([a-zA-Z]{3,})s+(\d{1,2}),s+(\d{4})\b/g, (match, monthEng, dayNum, year) =>
{
    const monthDe = monthMap[monthEng.toLowerCase()] || monthEng;
    return `${dayNum}. ${monthDe} ${year}`;
});

// "29 May"
text = text.replace(/\b(\d{1,2})s+([a-zA-Z]{3,})\b/g, (match, dayNum, monthEng) => {
    const monthKey = monthEng.toLowerCase();
    if (monthMap[monthKey]) {
        return `${dayNum}. ${monthMap[monthKey]}`;
    }
    return match;
});

// --- C. DYNAMISCHE SÄTZE ---
text = text.replace(/Selected flavors:s*(\s*(\d+)\s+flavors for\s+(\d+)\s+weeks\s*)\)/gi,
'Ausgewählte Sorten: ($1 Sorten für $2 Wochen)');

// --- D. EINFACHE ÜBERSETZUNGEN ---
for (const [english, german] of Object.entries(textTranslations)) {
    const regex = new RegExp(`\\b${english}\\b`, 'gi');
    text = text.replace(regex, german);
}

// --- DOM AKTUALISIEREN ---
if (text !== originalText) {
    node.nodeValue = text;
}
}

// 3. Rekursive Funktion zum Durchlaufen des DOMs
function walkDOM(node) {
    if (node.nodeType === 3) {
        if (node.parentNode && (node.parentNode.tagName === 'SCRIPT' ||
node.parentNode.tagName === 'STYLE')) {
            return;
        }
    }
}

```

```

        if (node.nodeValue.trim() !== "") {
            processTextNode(node);
        }
    } else {
        for (let i = 0; i < node.childNodes.length; i++) {
            walkDOM(node.childNodes[i]);
        }
    }
}

// 4. Initialer Durchlauf
walkDOM(document.body);

// 5. MutationObserver
const observer = new MutationObserver((mutations) => {
    mutations.forEach((mutation) => {
        if (mutation.type === 'childList' && mutation.addedNodes.length > 0) {
            mutation.addedNodes.forEach((newNode) => {
                walkDOM(newNode);
            });
        } else if (mutation.type === 'characterData') {
            processTextNode(mutation.target);
        }
    });
});

observer.observe(document.body, {
    childList: true,
    subtree: true,
    characterData: true
});

console.log("StayAI React-Optimized Translation Script geladen.");
})();

```



Dein Joker für die Präsentation:

Wenn sie dich nach deinen "Herausforderungen" fragen, dann erzähle genau diese Geschichte!

"Die größte Herausforderung war nicht die Übersetzung, sondern das Rendering-Verhalten von React. In JSX werden Elemente oft als `<span>{"€"}{price}</span>` geschrieben. React kompiliert das in zwei völlig isolierte Text-Nodes. Ein klassischer Regex schlägt hier fehl. Ich musste meinen Code so anpassen, dass er das DOM nach dem Euro-Zeichen scannt, sich über die

DOM-Tree-Navigation (nextSibling) den benachbarten Text-Node holt und beide synchron aktualisiert."

Das ist exakt das tiefe Frontend-Verständnis, das Firmen bei React Native / Web Entwicklern suchen. Probier das Skript direkt mal in der Konsole aus – es sollte die Zahl jetzt einwandfrei schlucken!